Parallel Programming HW3

113062531 楊峻豪

Implementation

1. CPU (hw3-1)

使用 Floyd Warshall Algorithm 來實作 hw3-1. 在計算上, 將要更新的矩陣切成 BxB 的 block, 並使用 OpenMP 對不同 block 平行化, 這讓每個 threads 能同時更新不同的 block.

2. Single GPU (hw3-2)

I、 Data

首先我將 data 切成 64 x 64 的 block 處理, 若無法整除則用 padding 的方式避免程式中出現過多的 if-else branch, 如下圖.

```
// pad n up to multiple of 64
const int B = 64;
n_pad = ( (n + B - 1) / B ) * B;
```

而 cpu 中有 32 x 32 個 threads, 因此每次一個 thread 要負責四筆 block 裡面的 data.

II、 Configuration

由於 shared memory 只有 48KB (49152 bytes), 因此我的 block factor 設為 64. 因為在 phase 2 和 phase 3 需要在 shared memory 中存入兩個 block 的 data, 這樣一共 32768 bytes (block factor * block factor * Int size * 2)不會超過上限.

```
__global__ void phase2(int r, int *Dist_GPU, int n) {
    __shared__ int s[2 * 64 * 64];  // first 64x64 for (i,k), second 64x64 for (k,j)
```

```
__global__ void phase3(int r, int *Dist_GPU, int n) {
    __shared__ int s[2 * 64 * 64];  // s for (i,k) and (k,j)
```

Block 的數量取決於 phase. 在 phase 1 只需要處裡 Pivot 因此只要一個 block. Phase 2 需要處理 pivot 以外水平及垂直的 block 因此需要 2*(round-1)個 block. Phase 3 要處裡 Phase 1 跟 phase 2 沒有處裡到的 block 共 round*round-2*(round-1)-1 個 block.

```
// phase 1: pivot block (r,r)
phase1<<<1, block>>>(r, Dist_GPU, n_pad);

// phase 2: tiles in pivot row & pivot column
phase2<<<dim3(2, rounds - 1), block>>>(r, Dist_GPU, n_pad);

// phase 3: remaining tiles
phase3<<<dim3(rounds - 1, rounds - 1), block>>>(r, Dist_GPU, n_pad);
```

每個 block 裡面都開 32x32 個 threads.

III、 Implementation

i. Phase 1

Phase 1 是要處理 pivot block 的 data. 首先會將 data 從 global memory 搬到 share memory. 並且每個 threads 根據 configuration 的設定要負責 4 筆 data 的搬運.

```
    __shared__ int s[64 * 64];

    int i = threadIdx.y;  // 0..31
    int j = threadIdx.x;  // 0..31

    int base = r << 6;    // r * 64

    int bi = base + i;
    int bj = base + j;

    // load 4 values into shared memory (forming the 64x64 tile)
    s[i * 64 + j]                 = Dist_GPU[(bi      ) * n + (bj      )];
    s[i * 64 + (j + 32)]          = Dist_GPU[(bi      ) * n + (bj + 32)];
    s[(i + 32) * 64 + j]          = Dist_GPU[(bi + 32 ) * n + (bj      )];
    s[(i + 32) * 64 + (j + 32)]   = Dist_GPU[(bi + 32 ) * n + (bj + 32)];
```

接著會開始 Blocked Floyd Warshal 運算, 每個 threads 一樣負

責四筆 data 並且不同 iterations 之間因為有 dependency 因此

需要用__syncthreads()同步.

```
for (int k = 0; k < 64; ++k) {
    __syncthreads();
    int s_ik     = s[i        * 64 + k];
    int s_i32k   = s[(i + 32) * 64 + k];

    // upper-left
    int via0 = s_ik + s[k * 64 + j];
    if (via0 < s[i * 64 + j]) s[i * 64 + j] = via0;

    // upper-right
    int via1 = s_ik + s[k * 64 + (j + 32)];
    if (via1 < s[i * 64 + (j + 32)]) s[i * 64 + (j + 32)] = via1;

    // lower-left
    int via2 = s_i32k + s[k * 64 + j];
    if (via2 < s[(i + 32) * 64 + j]) s[(i + 32) * 64 + j] = via2;

    // lower-right
    int via3 = s_i32k + s[k * 64 + (j + 32)];
    if (via3 < s[(i + 32) * 64 + (j + 32)]) s[(i + 32) * 64 + (j + 32)] = via3;
}
```

完成 pivot 內部的更新後最後再將 shared memory 內部的 data

搬回 global memory. 每個 threads 一樣負責四比 data.

```
// write back to global memory
Dist_GPU[(bi      ) * n + (bj      )] = s[i * 64 + j];
Dist_GPU[(bi      ) * n + (bj + 32)]  = s[i * 64 + (j + 32)];
Dist_GPU[(bi + 32 ) * n + (bj      )] = s[(i + 32) * 64 + j];
Dist_GPU[(bi + 32 ) * n + (bj + 32)]  = s[(i + 32) * 64 + (j + 32)];
```

ii. Phase 2

Phase 2 要處裡 pivot column 及 pivot row 的 data 因此每

個 block 需要的 share memory 會是兩個 block size. 每個

threads 需要 load 八筆 data

```
// load D(b_i, k) into s
s[i * 64 + j]                 = Dist_GPU[(base_i + i      ) * n + (base_k + j      )];
s[i * 64 + (j + 32)]          = Dist_GPU[(base_i + i      ) * n + (base_k + j + 32)];
s[(i + 32) * 64 + j]          = Dist_GPU[(base_i + i + 32 ) * n + (base_k + j      )];
s[(i + 32) * 64 + (j + 32)]   = Dist_GPU[(base_i + i + 32 ) * n + (base_k + j + 32)];

// load D(k, b_j) into s + 4096
int offset = 64 * 64;  // 4096
s[offset + i * 64 + j]                 = Dist_GPU[(base_k + i      ) * n + (base_j + j      )];
s[offset + i * 64 + (j + 32)]          = Dist_GPU[(base_k + i      ) * n + (base_j + j + 32)];
s[offset + (i + 32) * 64 + j]          = Dist_GPU[(base_k + i + 32 ) * n + (base_j + j      )];
s[offset + (i + 32) * 64 + (j + 32)]   = Dist_GPU[(base_k + i + 32 ) * n + (base_j + j + 32)];
```

接著一樣是進行 Floyd Warshal algorithm

```
#pragma unroll
for (int k = 0; k < 64; ++k) {
    int left0  = s[i        * 64 + k];
    int left32 = s[(i + 32) * 64 + k];

    int right0  = s[offset + k * 64 + j];
    int right32 = s[offset + k * 64 + (j + 32)];

    int via0 = left0 + right0;
    if (via0 < val0) val0 = via0;

    int via1 = left0 + right32;
    if (via1 < val1) val1 = via1;

    int via2 = left32 + right0;
    if (via2 < val2) val2 = via2;

    int via3 = left32 + right32;
    if (via3 < val3) val3 = via3;
}
```

並將結果寫回 global memory

```
// write back updated tile
Dist_GPU[(gi0      ) * n + (gj0      )] = val0;
Dist_GPU[(gi0      ) * n + (gj0 + 32)]  = val1;
Dist_GPU[(gi0 + 32 ) * n + (gj0      )] = val2;
Dist_GPU[(gi0 + 32 ) * n + (gj0 + 32)]  = val3;
```

iii. Phase 3

Phase 3 要處裡 phase 1 以及 phase 2 沒有處裡到的 block，一樣每

個 threads 去各自去將 data 從 global memory load 到 shared memory

```
// load D(by, r) → s
s[i * 64 + j]                 = Dist_GPU[(base_i + i      ) * n + (base_k + j      )];
s[i * 64 + (j + 32)]          = Dist_GPU[(base_i + i      ) * n + (base_k + j + 32)];
s[(i + 32) * 64 + j]          = Dist_GPU[(base_i + i + 32 ) * n + (base_k + j      )];
s[(i + 32) * 64 + (j + 32)]   = Dist_GPU[(base_i + i + 32 ) * n + (base_k + j + 32)];

// load D(r, bx) → s + offset
int offset = 64 * 64;
s[offset + i * 64 + j]                 = Dist_GPU[(base_k + i      ) * n + (base_j + j      )];
s[offset + i * 64 + (j + 32)]          = Dist_GPU[(base_k + i      ) * n + (base_j + j + 32)];
s[offset + (i + 32) * 64 + j]          = Dist_GPU[(base_k + i + 32 ) * n + (base_j + j      )];
s[offset + (i + 32) * 64 + (j + 32)]   = Dist_GPU[(base_k + i + 32 ) * n + (base_j + j + 32)];
```

接著一樣是進行 Floyd Warshal algorithm

```
#pragma unroll
for (int k = 0; k < 64; ++k) {
    int left0  = s[i        * 64 + k];
    int left32 = s[(i + 32) * 64 + k];

    int right0  = s[offset + k * 64 + j];
    int right32 = s[offset + k * 64 + (j + 32)];

    int via0 = left0 + right0;
    if (via0 < val0) val0 = via0;

    int via1 = left0 + right32;
    if (via1 < val1) val1 = via1;

    int via2 = left32 + right0;
    if (via2 < val2) val2 = via2;

    int via3 = left32 + right32;
    if (via3 < val3) val3 = via3;
}
```

結束後將結果從 shared memory 寫回 global memory

3. Multiple GPUs (hw3-3)

I、 Data

還是一樣將 data 用 64×64 的 block 進行切分；若 data 大小無法整除 64，則透過 padding 的方式補齊，避免在 kernel 中產生額外的條件分支。同時在多 GPU 的架構中，我再以 row-block 為單位將所有 blocks 平均分配給兩張 GPU，使每張 GPU 只負責其中一段 row 區域的計算。每個 block 一樣是由 32×32 threads 組成，因此每個 thread 會負責 block 內的四筆資料

II、Configuration

在 mulitple gpu 上每張 gpu 的 configuration 大致與 single gpu 的設定相同

III、 Implementation & Communication

Implementation 的部分大致與 single GPU 相同，不同的是在 multiple GPU 上，data 會根據 row-block 分配到不同 GPU 上，因此只要某張 GPU 擁有當前 pivot row，就用 cudaMemcpyPeer 將該 row 從這張 GPU 的 global memory 傳給另一張 GPU。

```
int ownsPivotRow = (r >= startBlock && r < startBlock + numBlocks);

if (ownsPivotRow) {
    // Send pivot row to the other device
    int other = 1 - tid;
    size_t rowOffset = (size_t)r * BLOCK_SIZE * n_padded;
    checkCuda(cudaMemcpyPeer(Dist_GPU[other] + rowOffset, other,
                             Dist_GPU[tid] + rowOffset, dev,
                             BLOCK_SIZE * n_padded * sizeof(int)),
              "cudaMemcpyPeer pivot row");
}
```

另外，在每個階段使用 #pragma omp barrier 確保兩張 GPU 都完成更新，才進入下一階段計算。

Profiling Results

我使用 c18.1 做為測資. 可以發現 phase 1 因為只要處理 pivot block 因此 SM efficency 非常低. 在 phase 2 跟 phase 3 以後 sm efficency 變高, 並且 share memory 的 throughput 大於 shared memory 表示大部分的計算在 shared memory 完成

```
Device "NVIDIA GeForce GTX 1080 (0)"
    Kernel: phase1(int, int*, int)
         47                    achieved_occupancy                      Achieved Occupancy    0.498017    0.498158    0.498084
         47                          sm_efficiency                 Multiprocessor Activity       4.40%       4.47%       4.45%
         47                  shared_load_throughput           Shared Memory Load Throughput  94.521GB/s  96.426GB/s  95.181GB/s
         47                 shared_store_throughput          Shared Memory Store Throughput  634.21MB/s  21.252GB/s  3.4243GB/s
         47                        gld_throughput                  Global Load Throughput  603.05MB/s  615.21MB/s  607.23MB/s
         47                        gst_throughput                 Global Store Throughput  603.05MB/s  615.21MB/s  607.23MB/s
    Kernel: phase2(int, int*, int)
         47                    achieved_occupancy                      Achieved Occupancy    0.858668    0.900706    0.882046
         47                          sm_efficiency                 Multiprocessor Activity      80.58%      84.85%      82.60%
         47                  shared_load_throughput           Shared Memory Load Throughput  2122.7GB/s  2366.0GB/s  2183.1GB/s
         47                 shared_store_throughput          Shared Memory Store Throughput  88.446GB/s  98.582GB/s  90.962GB/s
         47                        gld_throughput                  Global Load Throughput  132.67GB/s  147.87GB/s  136.44GB/s
         47                        gst_throughput                 Global Store Throughput  44.223GB/s  49.291GB/s  45.481GB/s
    Kernel: phase3(int, int*, int)
         47                    achieved_occupancy                      Achieved Occupancy    0.945968    0.950161    0.948107
         47                          sm_efficiency                 Multiprocessor Activity      98.54%      99.13%      98.92%
         47                  shared_load_throughput           Shared Memory Load Throughput  2836.4GB/s  2868.1GB/s  2853.1GB/s
         47                 shared_store_throughput          Shared Memory Store Throughput  118.18GB/s  119.50GB/s  118.88GB/s
         47                        gld_throughput                  Global Load Throughput  177.27GB/s  179.25GB/s  178.32GB/s
         47                        gst_throughput                 Global Store Throughput  59.091GB/s  59.751GB/s  59.439GB/s
```

Experiments & Analysis

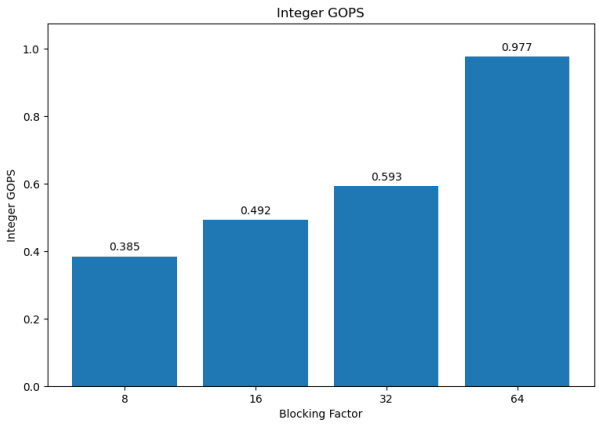
1. System Spec

使用課堂提供的 Apollo Server

2. Blocking Factor

以下的實驗數據來自 phase 3.

I、 Integer GOPS

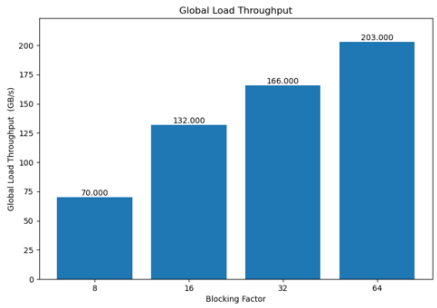


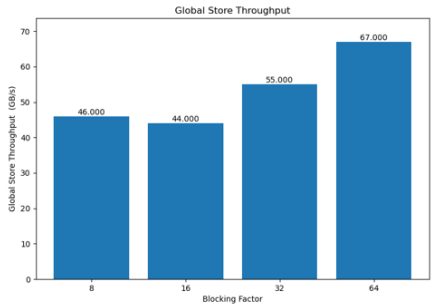
inst_integer 代表 GPU 執行的整數指令數，隨著 block factor 越大每個 block 所需執行的資料越多.

II、 Global Memory Bandwidth

Blocking Factor 越大， Load Throughput 持續提升。

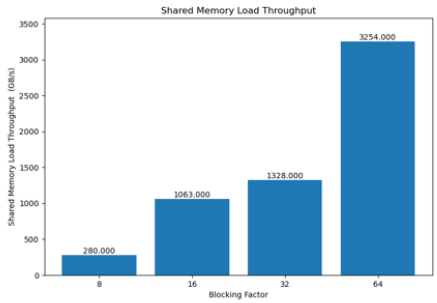
這是因為 tile 越大時，每個 block 一次會從 global memory 載入更多的資料到 shared memory. 並且因為主要的讀寫過程在 shared memory 完成因此 throughput 提升的幅度較緩和.

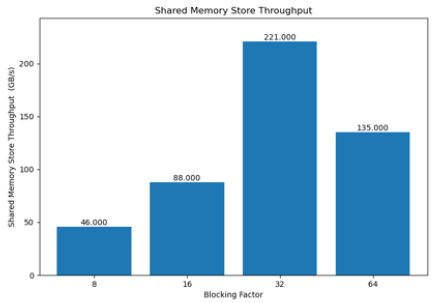




III、 Shared Memory Bandwidth

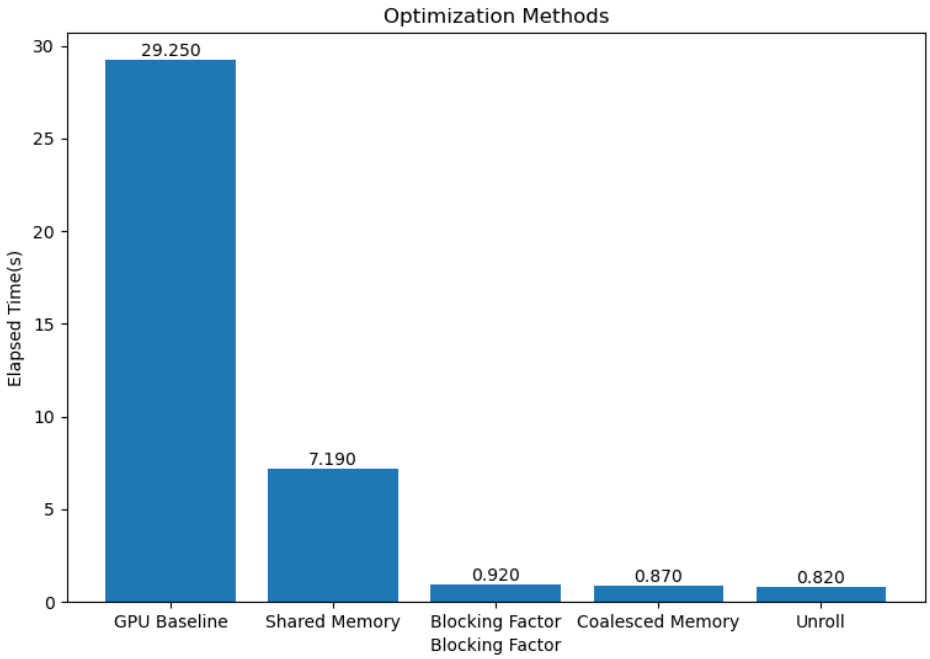
從圖中可以看到 Blocking Factor 越大, shared memory reuse 越高, kernel 會更多地從 shared memory 讀資料, 透過重複利用 tile 中的元素提升 load throughput。





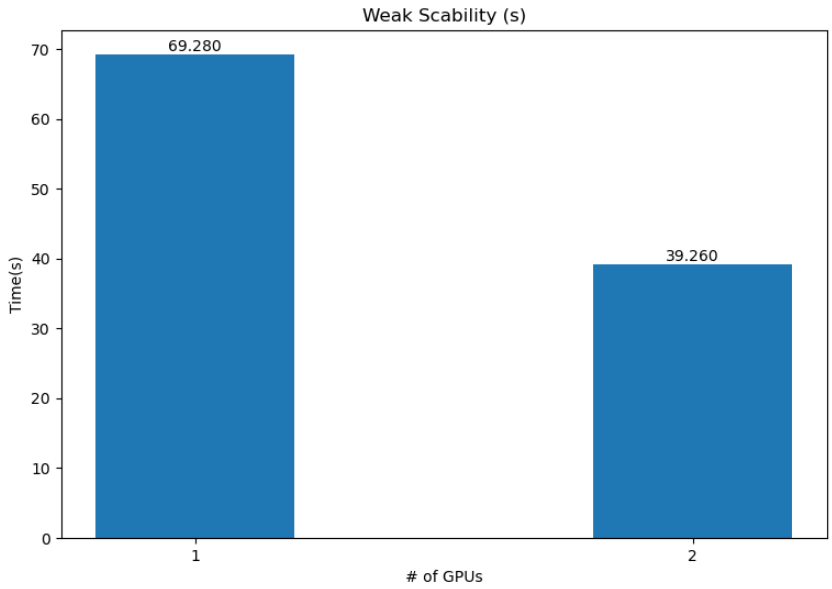
3. Optimization

實驗跑在 p11k1 這筆測資上. 根據實驗結果, 將資料載入 Shared Memory 與使用 Blocking 分塊計算是提升效能的主要因素, 可將執行時間從 29.25 s 降至 0.92 s. 進一步採用 Coalesced Memory 與 Loop Unroll 可略微提升效能至 0.82 ms.



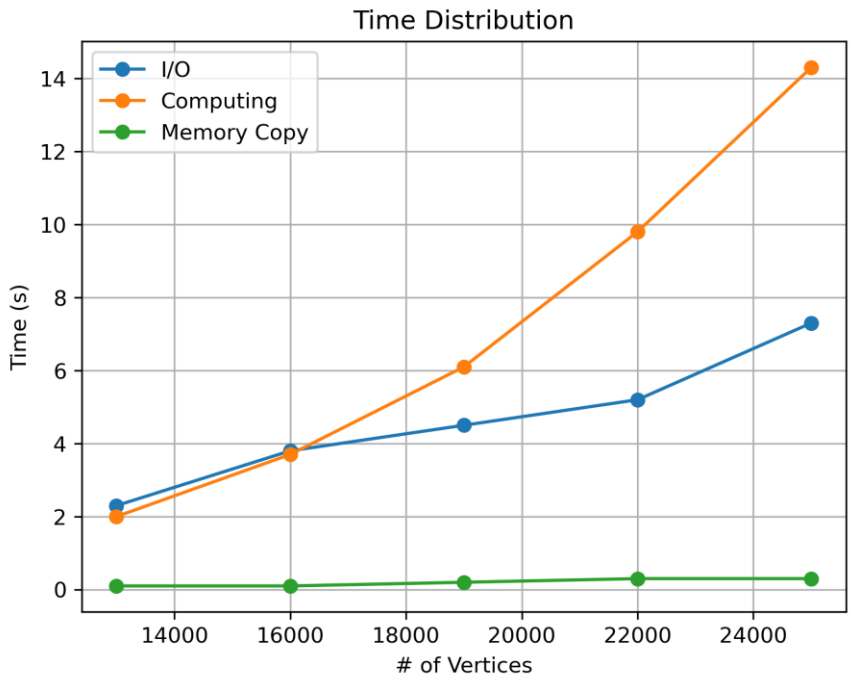
4. Weak Scalability

實驗跑在 c06.1 測資上, 當 GPU 數量從 1 增加到 2 時，運算時間從 69.28 ms 降至 39.26 ms，對 weak scalability 約 88%，顯示多 GPU 系統下演算法能有效保持近乎線性的性能增長，僅有少量因跨 GPU 通訊與同步造成的效率損失。



5. Time Distribution

根據實驗數據可以觀察到，在資料量(# of vertices)比較小的情況下整個運算會是 I/O bound. 隨著資料量變大逐漸變成 compute bound.



AMD GPU Porting and Analysis

1. 使用 hipify-clang

2. 在 AMD GPU 上，每個 wavefront 包含 64 個 threads，不同於 NVIDIA 的 warp（32 threads）。原本設計的 32×32 threads block（1024 threads）對 AMD 架構效率不佳。為了充分利用硬體，每個 block 的 thread 數設計為 wavefront 的整數倍，例如 256 threads，每個 thread 處理多個元素對應 tile 的大小。這樣可以確保每個 CU 的 wavefront 被完全調度，避免 thread 空閒浪費，提升運算效率。

Conclusion

這份作業讓我更清楚地了解 GPU 的內部架構與運作邏輯，並實際操作了不同

階段的 kernel（phase1~phase3）, 對計算與記憶體存取模式有更直觀的認識。為了提高效率, 我學會了使用 coalesced memory 並盡量避免 shared memory 的 bank conflict. 在實驗上, 透過觀察可以了解 block size、tile size 對執行效率的影響.